Name: Atharva Chaudhari
Roll No: 42

Experiment No. 10

1. Dataset Source:

The dataset used in this experiment is the MNIST Dataset.

🔗 Source link:
https://yann.lecun.com/exdb/mnist/

2. Dataset Description

The MNIST dataset is a collection of handwritten digit images used for image processing and machine learning tasks.

a) **Key Features:**

- **Images:** Grayscale images of digits (0–9)
- **Image Size:** 28 × 28 pixels
- **Pixel Range:** 0–255 (normalized to 0–1)
- **Training Samples:** 60,000 images
- **Testing Samples:** 10,000 images

b) **Target Variable:**

- Digits (0 to 9) → **Not used in this experiment**
- This is because Autoencoder is an **unsupervised learning algorithm**

c) **Important Characteristics:**

- Simple and clean dataset
- Widely used for benchmarking

- Suitable for image reconstruction tasks

3. Mathematical Formulation of the Algorithm

Autoencoder consists of two parts:

a) Encoder:

Transforms input into compressed representation:

$$z = f(x) = \sigma(Wx + b)$$

b) Decoder:

Reconstructs original image:

$$\hat{x} = g(z) = \sigma(W'z + b')$$

c) Loss Function:

The model minimizes reconstruction error:

$$L = -\sum [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

In your code:

- Loss used = Binary Cross Entropy
- Goal = Minimize difference between original and reconstructed image

4. Algorithm Limitations

Autoencoders have some limitations:

- Cannot generate completely new data like GANs
- Performance depends on architecture design
- May fail on very noisy or complex datasets
- Overfitting can occur if model is too deep
- Not suitable for classification directly.

5. Methodology / Workflow

Step-by-step Process:

1. **Load Dataset**
    - MNIST dataset is loaded from Keras
2. **Data Preprocessing**
    - Normalize pixel values (0–1)
    - Reshape images to (28, 28, 1)
3. **Add Noise**
    - Gaussian noise added to images
    - Creates noisy input for training

4. **Build Autoencoder**
    - Encoder → Conv + Pooling
    - Decoder → Upsampling + Conv
5. **Model Training**
    - Input: Noisy images
    - Output: Clean images
6. **Prediction**
    - Model reconstructs denoised images
7. **Visualization**
    - Original images
    - Noisy images
    - Denoised output

6. Performance Analysis

a) Evaluation Metric:

- Loss Function: Binary Cross Entropy

b) Observations:

- Training loss decreases over epochs
- Denoised images are clearer than noisy images
- Model successfully removes Gaussian noise

c) Interpretation:

- Autoencoder learns important features of images
- It reconstructs images by removing noise patterns

7. Hyperparameter Tuning

a) Parameters Used:

- Epochs = 5
- Batch Size = 128
- Noise Factor = 0.5
- Optimizer = Adam.

🔹 Tuning Process:

Parameter	Effect
Epochs ↑	Better learning but more time
Noise Factor ↑	Harder denoising task
Filters ↑	Better feature extraction
Batch Size ↓	Slower but more accurate

b) Impact:

- Increasing epochs improves reconstruction quality
- Proper noise level ensures effective learning
- Balanced architecture avoids overfitting

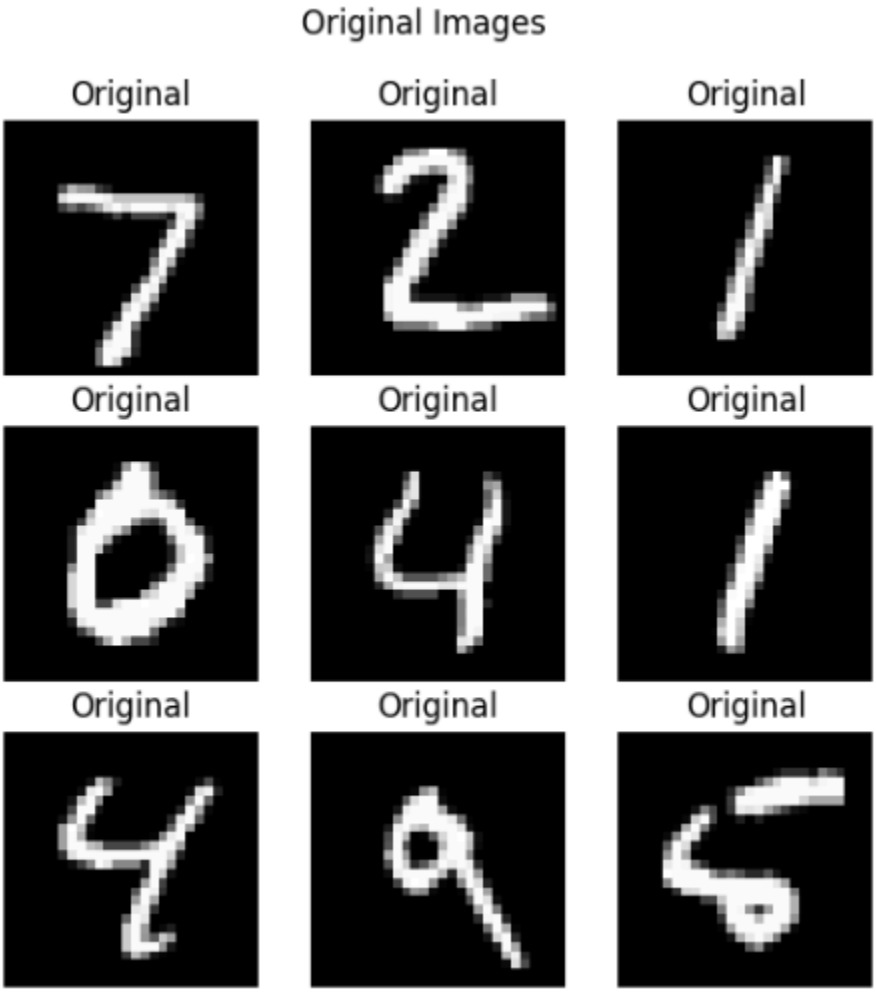
CODE:

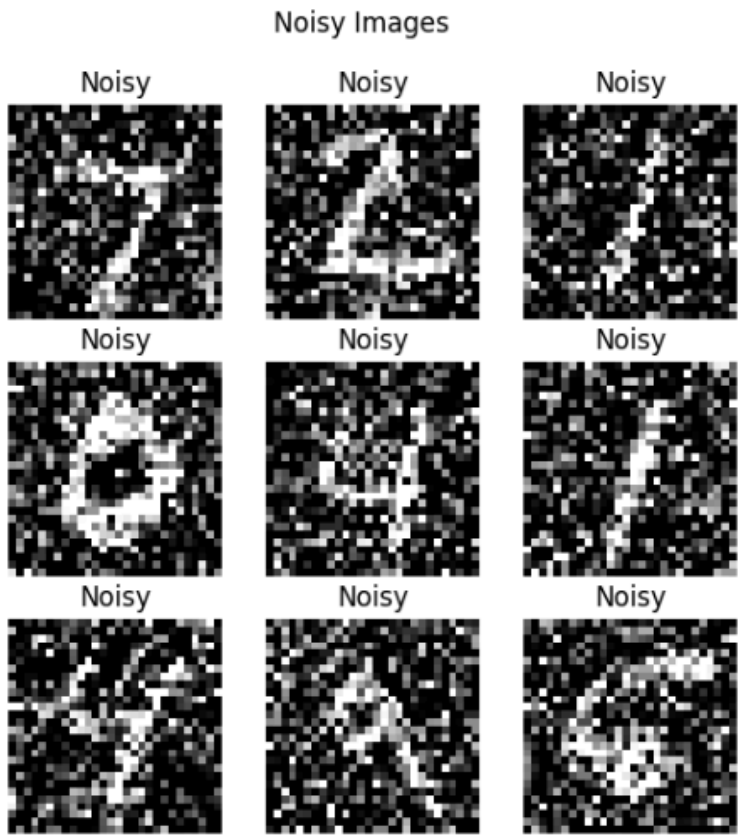
```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D

(x_train, _), (x_test, _) = mnist.load_data()

# Normalize
x_train = x_train.astype('float32') / 255.
x_test  = x_test.astype('float32') / 255.

# Reshape
x_train = x_train.reshape(-1, 28, 28, 1)
x_test  = x_test.reshape(-1, 28, 28, 1)

print("Dataset Loaded Successfully")

noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(size=x_train.shape)
x_test_noisy  = x_test + noise_factor * np.random.normal(size=x_test.shape)

# Clip values
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy  = np.clip(x_test_noisy, 0., 1.)

print("Noise Added")
```

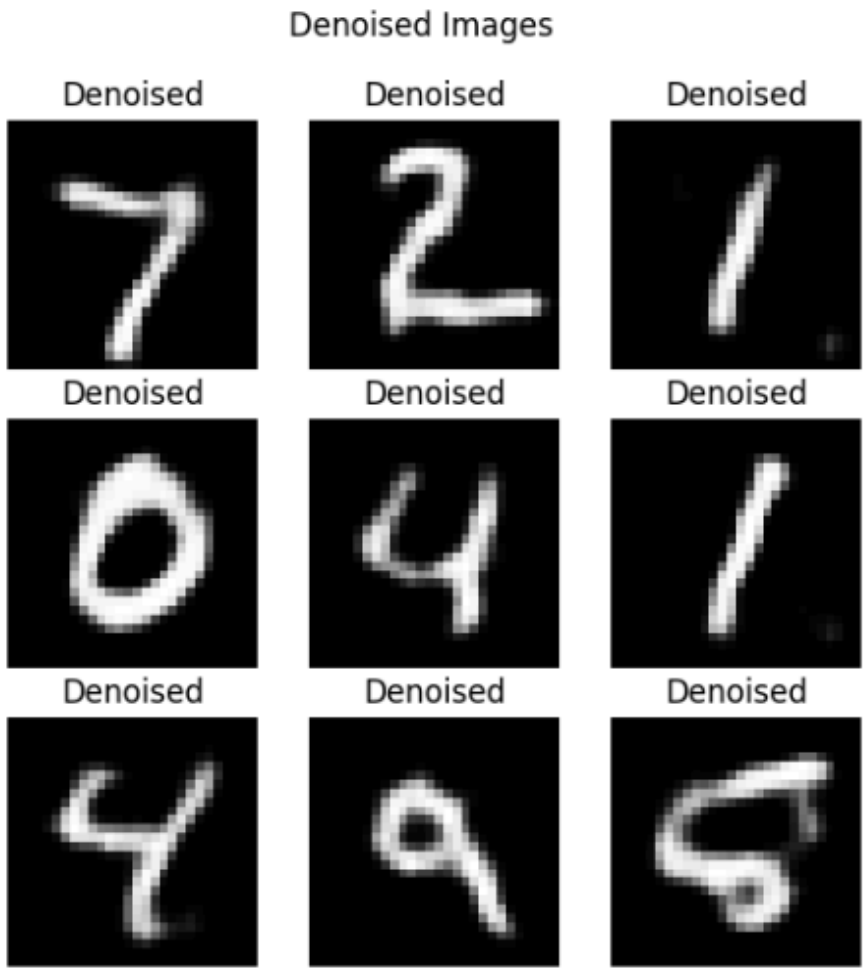
```
input_img = Input(shape=(28,28,1))

# Encoder
x = Conv2D(32, (3,3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)

x = Conv2D(16, (3,3), activation='relu', padding='same')(x)
encoded = MaxPooling2D((2,2), padding='same')(x)

# Decoder
x = Conv2D(16, (3,3), activation='relu', padding='same')(encoded)
x = UpSampling2D((2,2))(x)

x = Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = UpSampling2D((2,2))(x)

decoded = Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)

# Model
autoencoder = Model(input_img, decoded)

# Compile
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

print("Model Built Successfully")

history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=5,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

```
print("Training Completed")

decoded_imgs = autoencoder.predict(x_test_noisy)

print("Prediction Done")

plt.figure(figsize=(6,6))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title("Original")
    plt.axis('off')
plt.suptitle("Original Images")
plt.show()
plt.figure(figsize=(6,6))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')
plt.suptitle("Noisy Images")
plt.show()
plt.figure(figsize=(6,6))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')
plt.suptitle("Denoised Images")
plt.show()
```

OUTPUT:


Original Images
Original
Original
Original
Original
Original
Original
Original
Original
Original


Noisy Images
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy
Noisy


Denoised Images
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised
Denoised